NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Islamabad Campus

Design and Analysis of Algorithms | BS Data Science & Artificial Intelligence

ASSIGNMENT 2

Total: 100 marks

Submission Rules

- **Individual assignment.** Each student must complete and submit their own work.
- **Submission:** Upload the final assignment to GCR (Google Classroom): the PDF report, a ZIP of your source code and tests, and `results.csv`.
- **Submission deadline:** Late submissions will not be entertained.
- **Format:** Submit a typed PDF report of at most 5 pages, strictly enforced, with the naming convention RollNo-A2-Section, e.g., 23i-2605-A2-B. Name the code ZIP the same way.
- **Graphs:** Every graph must have labelled axes with units, a legend and a caption. Label every series with your own contender names (Insertion Sort, Merge Sort, Naive Quick Sort, Hardened Quick Sort, Heap Sort, size-K heap, ...), not generic labels such as "Algorithm 1".
- **Clashes:** If theory and measurement disagree, or two parts of the problem pull you in different directions, say so and choose. Leaving a clash unmentioned loses marks.
- **Stay inside this problem.** Do not add extra datasets, sorting libraries or machine-learning models to make the solution look more modern.
- A neat report with an empty or vague Prediction Log or Recommendation (Problem H) will receive no more than 40% of the total marks.
- **Plagiarism** will result in a zero for the entire assignment category. Sharing solutions across sections, or submitting results you have not verified by running your own code, will be handled according to the course policy.

The Midnight Queue

Ranking Suspicious Transactions at Scale

1. Overview

A payments processor scores card transactions around the clock. Its fraud analysts can investigate only **K transactions per shift**. Anything ranked below position K is never looked at, so the order of the queue decides which fraud is stopped and which gets through. The platform lead wants a ranking engine, and she wants it defended. She will not accept "it works". She wants to know which algorithm to ship, for which job, under which data conditions, and what breaks it. You are an engineer. Build the engine, run the experiments, derive the mathematics, and write a short recommendation she can act on.

2. Learning Objectives

On completing this assignment you should be able to:

1. Select among Insertion, Merge, Quick and Heap Sort on the basis of data properties rather than habit.
2. Derive and solve recurrences using the iteration, recursion-tree and Master's methods.
3. Decide when a full sort is unnecessary, as in top-K selection and incremental updates.
4. Compare predicted growth with measured running time, and explain every gap.

3. Data and Definitions

3.1 Dataset

- **Dataset:** Credit Card Fraud Detection (ULB Machine Learning Group), hosted on Kaggle
- **Link:** https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- **File:** `creditcard.csv`, containing 284,807 transactions made by European cardholders over two days, of which 492 are labelled fraud
- **Columns:** `Time` (seconds since the first transaction), `V1` to `V28` (anonymised numeric features), `Amount`, `Class` (1 = fraud)

Download the file yourself with a free Kaggle account. **Do not include the dataset in your submission.** Check the licence terms on the Kaggle page.

3.2 Definitions (fixed for all students)

- **txn_id:** the row number in the file, starting at 1.
- **Standardised features:** $z_i = (V_i - \text{mean}_i) / \text{std}_i$, computed over the full file for $i = 1..28$.
- **Risk score:** $\text{risk} = z_1^2 + z_2^2 + \ldots + z_{28}^2$.
- **Risk band:** $\text{band} = \min(9, \max(0, \text{floor}(\log_2(\text{risk}))))$.
- **Triage order (the sort key):** band descending, then Amount descending, then Time ascending, then txn_id ascending. This is a total order, so every correct sort must produce the same output.

`Class` is for **evaluation only**. It must never appear in the risk score or in any sort key. No machine-learning model is to be trained or used anywhere in this assignment. NumPy and pandas may be used for loading the data and computing the risk score.

3.3 Personal parameters

Let **s** be your roll number (digits only).

- **K = 25 + (s mod 226):** your shift capacity.
- All random choices (sampling, shuffling, displacement) use seed **s**. Subsamples of size n are drawn uniformly without replacement and then restored to Time order.

Results produced with another student's parameters will not match yours.

3.4 Input conditions

Every experiment must be run on the conditions below, each built from the same records.

ID	Condition	Construction
C1	Arrival	Records in file order (Time ascending)
C2	Shuffled	C1 shuffled with seed s
C3	Pre-ranked	Records already in triage order
C4	Reverse-ranked	Exact reverse of triage order
C5	Nearly ranked	C3 with 1% of records moved to random positions
C6	Band-only	C1, but the sort key is `band` alone

Sizes: n = 1,000; 10,000; 100,000; 284,807 (full file).

4. Ground Rules

1. Implement Insertion Sort, Merge Sort, Quick Sort and Heap Sort **yourself**. No built-in sort may appear inside them. `sorted()` or NumPy sorting may be used only to verify correctness and as a clearly labelled baseline.
2. Every contender receives a fresh copy of identical input.
3. Count **key comparisons** and **element moves** with one consistent definition, stated in your report.
4. Time with `perf_counter` (or your language's equivalent). Exclude file reading and risk-score computation. Report the median of 5 runs. State your hardware and software.
5. **DNF rule:** any single run that exceeds 5 minutes is recorded as DNF. Report the largest n that completed and **predict** the missing point from your own analysis. The prediction is graded against what you later measure on a smaller scale, not against nothing.
6. Any programming language is allowed. A user interface is not required, and the only interface credit available is the optional bonus in Section 6.

5. Problems

Complete all eight problems. The marks for each are shown in its heading.

Problem A: Know Your Data (5 marks)

Profile the dataset before touching an algorithm. Report:

- the distribution of risk bands (table and histogram);
- the number of distinct `Amount` values and the share held by the 10 most frequent;
- the number of frauds and how they are spread across bands;
- which of the properties in Section 3.4 the raw file already exhibits, with evidence.

Problem B: Build the Contenders (20 marks)

Implement the following algorithms. Marks are awarded for a correct, tested implementation of each.

- Insertion Sort (3 marks)
- Merge Sort (4 marks)
- Quick Sort (5 marks): a **naive** version (last element as pivot, Lomuto partition) and one **hardened** variant of your choice (random pivot, median-of-three, or three-way partition)
- Heap Sort (4 marks)
- Top-K selection (4 marks): a **size-K min-heap** and a **bounded insertion buffer** of size K

Test against a trusted reference on every condition and size, and on: empty input, a single record, all-equal keys, and duplicate-heavy input. Because the triage order is total, outputs must be identical to the reference, not merely sorted.

If a contender fails or crashes on some input (for example, from recursion depth), document it and state how you handled it.

Problem C: The Full Ranking Tournament (10 marks)

Run all five full-sort contenders on every combination of condition and size.

Before each round, write a **prediction** in your prediction log: which contender will win, which will lose, and why. After the run, record what you got wrong.

Deliver:

1. Execution time vs n for all contenders on C2 (log-log).
2. Execution time at n = 10,000 across all six conditions (grouped bars).
3. Comparisons and moves vs n for each contender on C2.
4. Time divided by n log n and by n^2 vs n, for each contender.
5. A summary table of time, comparisons, moves, extra memory and stability for every contender and condition.

Problem D: The Shift Queue (7 marks)

Analysts need only the top K. Compare three ways to produce them:

- (a) full sort, then take the first K;
- (b) the size-K min-heap;
- (c) the bounded insertion buffer.

Run each for $K \in \{10, 50, \text{your K}, 500, 5{,}000, n/10\}$ at n = 100,000 and 284,807, on C1 and C2.

Then answer from your data:

- Where does each method stop being the best choice?
- Using `Class` for evaluation only, plot the fraction of all frauds captured in the top K as K grows. What is the smallest K that captures half of them under this scoring rule, and what does it cost to produce that queue by each method?

Problem E: The Live Feed (7 marks)

Transactions arrive over 48 hours. Split the file into consecutive windows by `Time`. After each window, publish the top K among **all transactions seen so far**.

Compare:

- (a) re-rank everything after every window;
- (b) maintain a size-K heap fed only by the new window's records;
- (c) Merge Sort the new window, then merge it into the current top-K list.

Run with **1-hour windows** and with **10-minute windows**. Plot cumulative time against window number for each strategy and report peak extra memory.

Problem F: The Mathematical Case (22 marks)

Answer each item with a derivation, using the stated method.

Item	Question	Method	Marks
M1	Insertion Sort: best, average and worst case. Express the running time using n and the number of inversions I. Count I on your C5 sample.	Direct analysis	3
M2	Merge Sort: solve $T(n) = 2T(n/2) + \Theta(n)$.	Iteration, then verify with Master's theorem	3
M3	Quick Sort. (a) Worst case: $T(n) = T(n-1) + \Theta(n)$, by iteration. (b) Best case: $T(n) = 2T(n/2) + \Theta(n)$, by recursion tree. (c) A 1:9 split: $T(n) = T(n/10) + T(9n/10) + n$, by recursion tree. Give a tight bound for (c).	Iteration and recursion tree	4
M4	Heap Sort: justify the cost of building the heap, and the total cost.	Analysis	3

Item	Question	Method	Marks
M5	Top-K: derive the cost of methods (a), (b) and (c) from Problem D in terms of n and K. Predict the crossover K for n = 100,000, then compare with your measurement.	Derivation and prediction	3
M6	Naive Quick Sort on C6: let the band groups have sizes $m_1 \ldots m_d$. Derive its comparison count in terms of these sizes, compute the prediction from your actual groups, and compare with the measured count.	Derivation and comparison	3
M7	Live feed: derive the total cost of strategies (a), (b), (c) from Problem E over w windows in terms of n, w and K. Find when the best strategy changes.	Derivation	2
M8	Space complexity of every contender.	Analysis	1

Problem G: Change Request (4 marks)

A change request from the platform lead will be posted on the course portal **48 hours before the deadline**. Add a **short addendum section to your report**: what changed, which of your earlier conclusions still hold and which do not, with fresh measurements.

Problem H: The Recommendation (5 marks)

Write a short report section for the platform lead, not for your instructor. For each job below, state the algorithm or strategy you would ship:

1. the overnight batch that ranks a full day of transactions;
2. the shift queue of K cases;
3. the live refresh every window.

For each choice, give the evidence from your own data and mathematics, state the assumption it depends on, and name one condition under which your recommendation would be wrong.

6. Bonus: The Analyst Console (5 marks, optional)

Build a lightweight interface (web page or desktop) through which an analyst can choose the algorithm that builds the queue, set K, and view the top K transactions with their risk band, Amount and Time. The interface should also display the running time and comparison count of each run.

- Bonus marks are assessed only if Problems A to H are complete.
- The interface is judged on how clearly it exposes algorithmic behaviour, not on visual polish. A polished interface cannot compensate for missing algorithmic work.
- Describe it in one short paragraph of your report and demonstrate it in the viva.
- Bonus marks are added to your total, to a maximum of 105.

7. Deliverables

1. Source code with tests and a README that reproduces your results from the Kaggle file.
2. `results.csv` containing every measurement.
3. Report in PDF (maximum 5 pages, strictly enforced) covering Problems A to H, including the change request addendum, the recommendation and a short AI-use disclosure paragraph (see Section 9), plus the prediction log as an appendix.
4. Live demonstration and viva.

8. Assessment

Part	Algorithmic focus	Marks
A	Data profile: distributions, ties and ordering	5
B	Implementation and correctness: Insertion 3, Merge 4, Quick 5, Heap 4, Top-K 4	20
C	Full ranking tournament: sorting under six input conditions	10
D	Shift queue: top-K selection against full sorting	7
E	Live feed: incremental against repeated ranking	7
F	Mathematical case: recurrences and asymptotic analysis	22
G	Change request: adapting a design to new constraints	4
H	Recommendation: algorithm selection justified by evidence	5

Part	Algorithmic focus	Marks
Viva	Oral examination on your own algorithms and results	20
Total		**100**
Bonus	Analyst Console (Section 6)	5

Viva. You will be asked to predict how your algorithms behave on an unseen input, to modify your own code, and to justify your results. A viva score below 50% caps every other component at 60% of its marks.

9. Academic Integrity and AI Use

AI tools are permitted for code, plots and drafting. You must disclose which tools you used and for what, in one paragraph.

Marks reward understanding. Anything you cannot explain or modify live in the viva loses marks, however correct it is. Copying results or prediction logs from another student is plagiarism.

Y’all are cooked. Best of luck :)